Лабораторная работа №5

«Разметка изображений и видеокадров производственных объектов»

Цель работы: приобрести навыки формирования набора данных и разметки изображений производственных объектов, освоить форматы хранения аннотаций и разделение набора на обучающую, валидационную и тестовую выборки, а также научиться программно проверять и анализировать качество разметки на Python.

Теоретические сведения

Метод «обучение с учителем» основан на том, что модель обучается на примерах, для каждого из которых заранее известен правильный ответ: на этом изображении находится деталь без дефекта, на этом – деталь с дефектом. Данные, в которых содержится информация о целевом признаке, называются размеченными. Разметка – как правило, сложный и длительный процесс, если его нельзя автоматизировать, а от её качества напрямую зависит качество финальной модели: ошибка в разметке для нейронной сети является «правильным ответом», которому она будет следовать.

Вид разметки определяется решаемой задачей (лабораторная работа №4). Для классификации метка присваивается изображению целиком, для детектирования – каждому объекту присваивается класс и ограничивающая рамка (bounding box), для сегментации – объект обводится полигоном по контуру. Чем подробнее разметка, тем она трудоёмче, поэтому следует выбирать минимально достаточный для задачи вид разметки.

Источники данных и подготовка кадров

Набор данных можно собрать самостоятельно в виде набора изображений или видеозаписей (при этом объект должен быть чётко виден) либо взять открытый набор: Kaggle Datasets, Roboflow Universe, ImageNet, MS COCO, Open Images. Видеозапись предварительно разделяют на отдельные кадры с помощью утилиты ffmpeg. Команда ниже сохраняет два кадра в секунду в папку frames (папку необходимо создать заранее):

```
mkdir frames
ffmpeg -i video01.mp4 -vf fps=2 frames/video01_%04d.jpg
```

Частоту кадров выбирают так, чтобы соседние сохранённые кадры заметно отличались: на практике кадры видеопотока почти одинаковы, и набор из таких кадров мало добавляет к обучению. Именно поэтому в набор необходимо включать съёмку при разном освещении, с разных ракурсов, на разном фоне и расстоянии. Если набор состоит из соседних кадров, снятых в одинаковых условиях, нейронная сеть может «выучить» набор данных вместо того, чтобы выделить обобщающие признаки (переобучение). Имя файла

кадра рекомендуется начинать с имени ролика (video01_0001.jpg) – это понадобится при разбиении набора на выборки.

Правила маркировки изображений

Качество меток напрямую влияет на точность обученной модели. При разметке рекомендуется соблюдать следующие правила.

Правило 1. Размечать каждый интересующий объект на каждом изображении. Модель учится находить шаблоны пикселей, соответствующие объекту. Если объект на части изображений остался без метки, для модели он выглядит как фон, и в обучение вносятся ложные отрицательные примеры.

Правило 2. Обозначать объект целиком. Рамка должна охватывать весь объект: разметка только части объекта сбивает модель с толку относительно того, что представляет собой полный объект.

Правило 3. Делать рамки узкими. Рамка должна плотно прилегать к объекту, но не обрезать его часть. Слишком широкая рамка включает фон и учит модель считать фон частью объекта, а слишком узкая – теряет признаки самого объекта (рисунок 1).

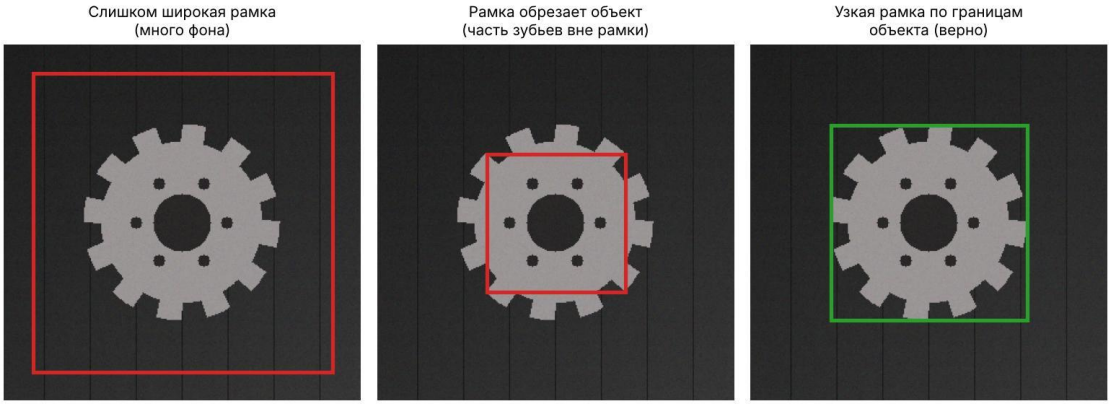


Рисунок 1 – Варианты ограничивающей рамки: слишком широкая и обрезающая объект рамки недопустимы, верной является рамка, плотно прилегающая к границам объекта

Правило 4. Размечать скрытые объекты. Окклюзия – ситуация, когда объект частично закрыт другим объектом. Закрытый объект всё равно размечают, причём так, как если бы он был виден полностью, а не рисуют рамку только вокруг видимой части. Рамки при этом могут перекрываться, это допустимо (рисунок 2).

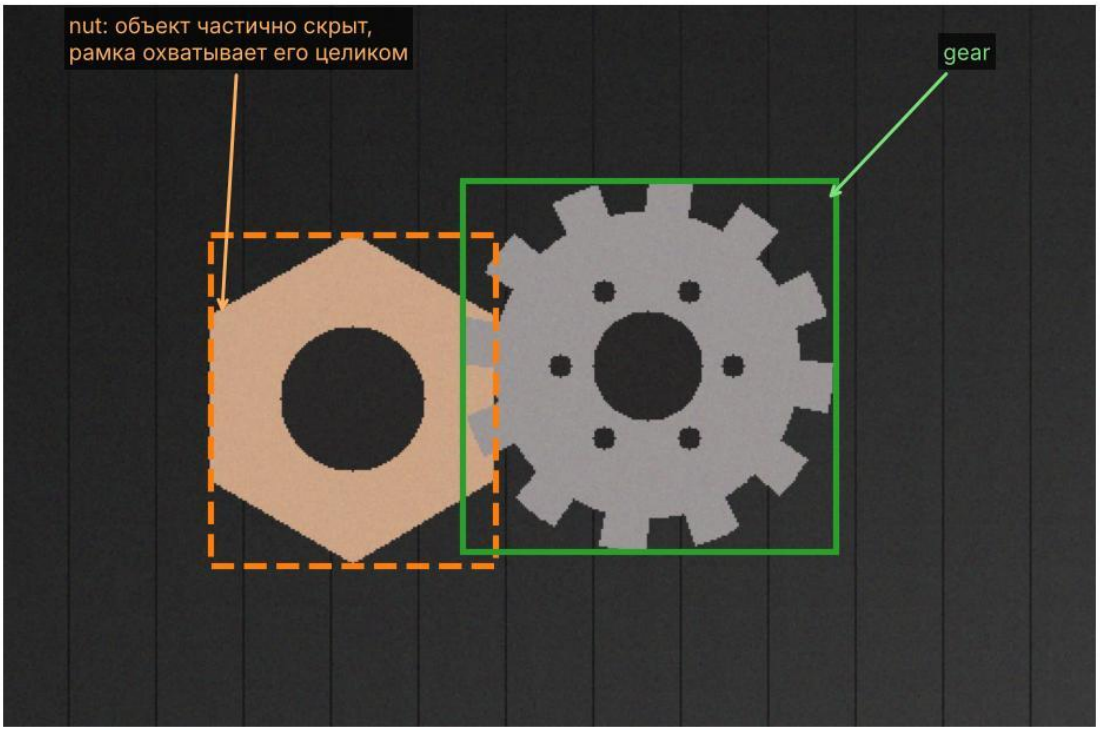


Рисунок 2 – Разметка перекрытых объектов: рамка гайки охватывает её целиком, хотя часть гайки закрыта шестерёнкой

Правило 5. Использовать конкретные имена классов. Лучше ошибиться в сторону большей конкретности: несколько узких классов всегда можно объединить в один общий (например, bolt_m6 и bolt_m8 в bolt), тогда как разделить общий класс на узкие без повторной разметки нельзя. Имена классов задают латиницей без пробелов (gear, washer, nut), а не на русском или в виде невыразительных названий.

Форматы хранения разметки

Результат разметки хранится в файлах аннотаций. В формате YOLO, который используется для обучения моделей YOLOv8 в лабораторной работе №6, каждому изображению соответствует текстовый файл с тем же именем. Каждая строка файла описывает один объект и содержит пять чисел: номер класса и четыре нормированных параметра рамки – координаты её центра и размеры. Нормирование выполняется делением на ширину W и высоту H изображения, поэтому все значения лежат в диапазоне от 0 до 1:

$$x_c = (x_1 + x_2) / (2 \cdot W), \quad y_c = (y_1 + y_2) / (2 \cdot H)$$

$$w = (x_2 - x_1) / W, \quad h = (y_2 - y_1) / H$$

где (x1, y1) и (x2, y2) – координаты левого верхнего и правого нижнего углов рамки в пикселях. Номера классов соответствуют порядку имён в файле описания набора (data.yaml). Нормированная запись не зависит от разрешения: если изображение растянуто до другого размера, значения в файле не изменяются (рисунок 3).

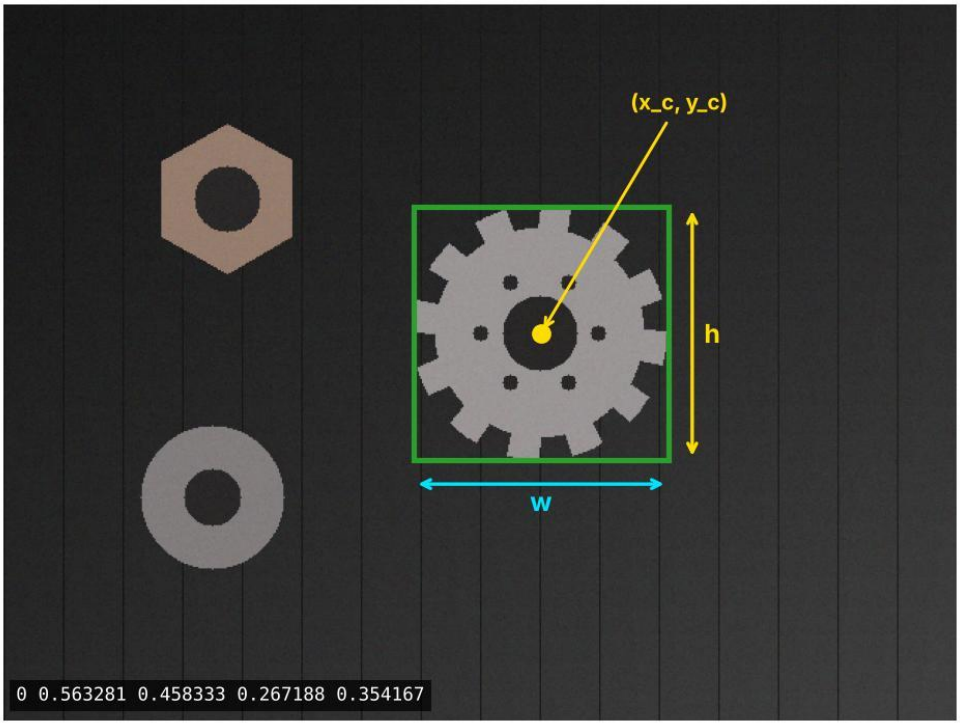


Рисунок 3 – Параметры рамки в формате YOLO: центр (x_c, y_c), ширина w и высота h; внизу – соответствующая строка файла аннотаций (класс 0 – gear)

Помимо YOLO, существуют и другие форматы аннотаций (Pascal VOC в виде XML, COCO в виде JSON); они отличаются способом записи, но содержат ту же информацию, и Roboflow при экспорте набора данных преобразует разметку в нужный формат. Для сегментации в формате YOLO строка содержит номер класса и последовательность нормированных координат вершин полигона: «класс x1 y1 x2 y2 … xn yn». Полигон точнее описывает форму объекта, чем рамка, но его разметка заметно трудоёмче (рисунок 4).

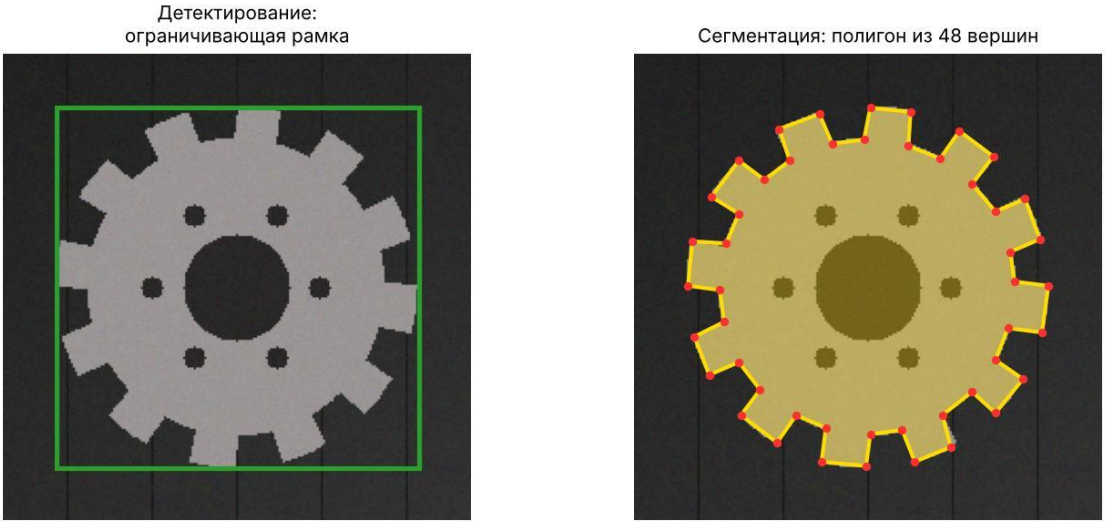


Рисунок 4 – Разметка одного и того же объекта рамкой и полигоном: рамка включает участки фона между зубьями, полигон повторяет форму детали

Выбор между рамкой и полигоном определяется задачей. Если достаточно знать, есть ли объект и где он находится, применяют рамки; если необходимо измерять форму, площадь или положение контура (например, площадь дефекта), применяют полигоны.

Разделение набора данных на выборки

Размеченный набор делится на три части. Обучающая выборка (train) – данные, на которых модель непосредственно обучается. Валидационная выборка (valid) нужна для проверки того, что модель научилась выделять признаки, а не запомнила обучающие примеры, и для подбора гиперпараметров. Тестовая выборка (test) используется один раз для итоговой оценки качества модели на данных, которых она не видела. Типичное соотношение – 60–80 % для обучающей, 10–20 % для валидационной и 10–20 % для тестовой выборки; например, набор из 1000 образцов делят на 800, 100 и 100. Данные разделяются случайным образом, чтобы выборки были репрезентативными.

Для кадров, полученных из видеозаписей, у случайного разбиения есть недостаток. Соседние кадры одного ролика почти одинаковы, поэтому при случайном распределении по кадрам почти точные копии обучающих изображений попадают в валидационную и тестовую выборки, и оценка качества получается завышенной. Надёжнее делить набор по роликам: все кадры одного ролика остаются в одной выборке (рисунок 5).

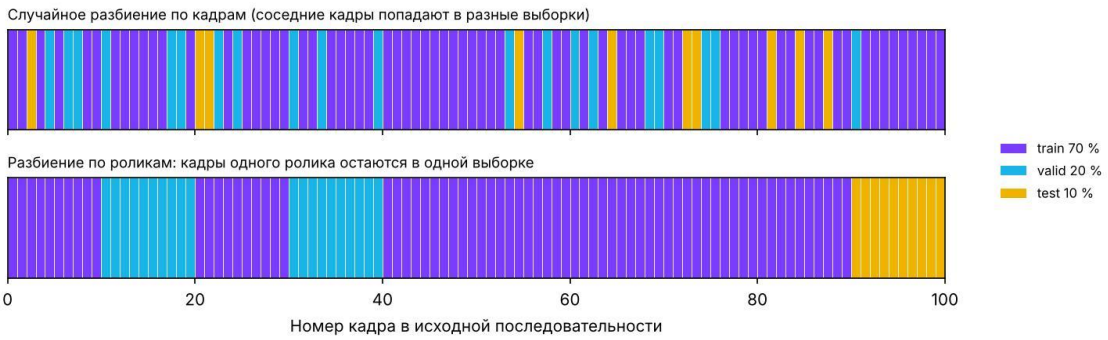


Рисунок 5 – Два способа разбиения 100 кадров на выборки в пропорции 70/20/10: случайное по кадрам и по роликам (по 10 кадров в ролике)

Кроме объёма выборок, важен баланс классов. Если один класс представлен значительно меньшим числом примеров, модель хуже его распознаёт. Баланс оценивают коэффициентом дисбаланса – отношением числа рамок самого частого класса к числу рамок самого редкого:

$$K_д = N_{max} / N_{min}$$

В данной работе в качестве ориентира принимается Кд ≤ 3. При большем значении набор дополняют кадрами с редким классом, применяют аугментацию или объединяют близкие классы (рисунок 6).

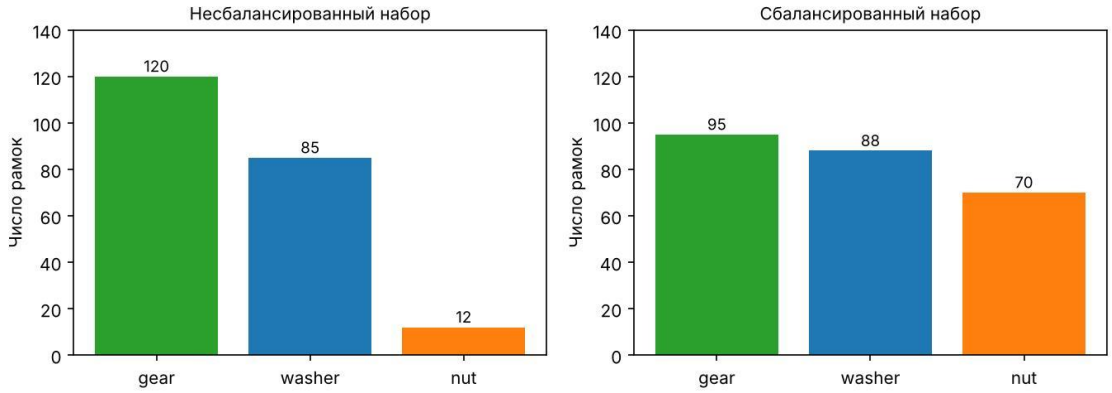


Рисунок 6 – Число рамок по классам: слева Кд = 10 (класс nut представлен недостаточно), справа Кд ≈ 1,4

Пример на рисунке 6 демонстрационный: числа выбраны для иллюстрации и не относятся к реальному набору данных.

Алгоритмы проверки и анализа разметки (индивидуальное задание)

Разметка выполняется вручную, поэтому неизбежно содержит ошибки: пропущенные объекты, случайные клики, рамки, вышедшие за границы кадра. Чтобы найти их до обучения, разметку проверяют программно. В настоящей работе рассматриваются три алгоритма: проверка и визуализация разметки, анализ полигонов сегментации и разбиение набора на выборки с анализом баланса классов. Каждый студент реализует только один алгоритм – в соответствии с номером своего варианта (раздел «Варианты заданий»). Все листинги проверены на демонстрационном наборе из 12 кадров с тремя классами объектов (gear, washer, nut).

Алгоритм 1. Проверка и визуализация разметки

Алгоритм читает файл аннотаций, проверяет корректность каждой строки и возвращает рамки на изображение. Для этого нормированные значения переводят обратно в пиксели:

$$x_1 = (x_c - w / 2)\cdot W, \quad x_2 = (x_c + w / 2)\cdot W$$

$$y_1 = (y_c - h / 2)\cdot H, \quad y_2 = (y_c + h / 2)\cdot H$$

Строка считается корректной, если в ней пять значений, номер класса существует, а все четыре параметра рамки лежат в диапазоне от 0 до 1. Листинг ниже выполняет проверку и рисует рамки с названиями классов (результат показан на рисунке 7):

```
import cv2
from pathlib import Path
```

```
names = ["gear", "washer", "nut"]

def check_label_line(line, n_classes):
    # проверка одной строки разметки в формате YOLO
    parts = line.split()
    if len(parts) != 5:
        return "ошибка: должно быть 5 чисел"
    cls = int(parts[0])
    vals = [float(v) for v in parts[1:]]
    if not 0 <= cls < n_classes:
        return "ошибка: неизвестный класс"
    if any(v < 0 or v > 1 for v in vals):
        return "ошибка: значение вне [0, 1]"
    return "ok"



img = cv2.imread("images/frame_000.jpg")
h, w = img.shape[:2]
lines = Path("labels/frame_000.txt").read_text().splitlines()

for line in lines:
    print(line, "->", check_label_line(line, len(names)))
    cls, xc, yc, bw, bh = line.split()
    xc, bw = float(xc) * w, float(bw) * w
    yc, bh = float(yc) * h, float(bh) * h
    x1, y1 = int(xc - bw / 2), int(yc - bh / 2)
    x2, y2 = int(xc + bw / 2), int(yc + bh / 2)
    cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(img, names[int(cls)], (x1, y1 - 6),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

cv2.imwrite("check_frame_000.jpg", img)
```

Если разметка корректна, для каждой строки программа выводит «ok», а на изображении рамки должны совпадать с границами объектов.

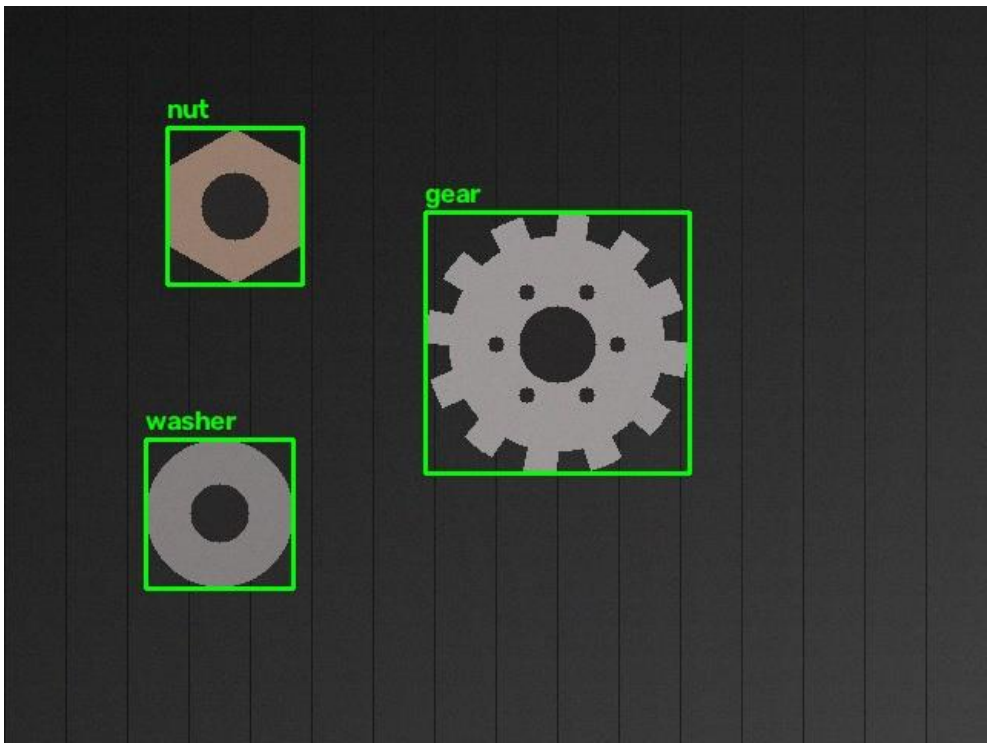


Рисунок 7 – Результат работы листинга: рамки, восстановленные из файла аннотаций, с названиями классов

Визуальная проверка находит не все ошибки, особенно в большом наборе, поэтому к ней добавляют автоматические проверки качества разметки. Проверка (а): слишком малая рамка – площадь w·h меньше заданной доли кадра (например, 0,1 %); такая рамка чаще всего возникает из-за случайного щелчка. Проверка (б): рамка выходит за границы кадра. Проверка (в): пустой файл разметки – если на изображении объект должен быть, это пропущенная разметка; пустой файл допустим только для изображений, на которых нет ни одного объекта интереса. Проверка (г): дубликат – две рамки одного класса, у которых центры и размеры почти совпадают.

Все четыре проверки реализованы в одной функции, которая просматривает каталог аннотаций и возвращает список замечаний:

```
from pathlib import Path


MIN_AREA = 0.001   # минимальная доля площади кадра
EPS = 0.005        # допуск совпадения рамок (дубликаты)
TOL = 1e-4         # допуск округления координат



def audit_labels(label_dir, n_classes):
    problems = []
    for path in sorted(Path(label_dir).glob("*.txt")):
```

```
        rows = [ln.split() for ln in path.read_text().splitlines()
                if ln.strip()]
        if not rows:                          # проверка (в)
            problems.append((path.name, "пустой файл разметки"))
        seen = []
        for r in rows:
            cls = int(r[0])
            xc, yc, bw, bh = map(float, r[1:5])
            if cls >= n_classes:
                problems.append((path.name, "неизвестный класс"))
            if bw * bh < MIN_AREA:            # проверка (а)
                problems.append((path.name, "слишком малая рамка"))
            x1, x2 = xc - bw / 2, xc + bw / 2
            y1, y2 = yc - bh / 2, yc + bh / 2
            if min(x1, y1) < -TOL or max(x2, y2) > 1 + TOL:  # (б)
                problems.append((path.name, "рамка вышла за кадр"))
            for s in seen:                    # проверка (г)
                diff = max(abs(xc - s[1]), abs(yc - s[2]),
                           abs(bw - s[3]), abs(bh - s[4]))
                if cls == s[0] and diff < EPS:
                    problems.append((path.name, "дубликат рамки"))
            seen.append((cls, xc, yc, bw, bh))
    return problems



for name, text in audit_labels("labels", 3):
    print(name, "-", text)
```

Работа функции проверена на копии демонстрационного набора, в которую намеренно внесены четыре ошибки – по одной каждого вида. Программа нашла все четыре и указала файлы, в которых они находятся:

```
frame_003.txt - пустой файл разметки
frame_004.txt - слишком малая рамка
frame_005.txt - рамка вышла за кадр
frame_006.txt - дубликат рамки
```

Студенты, выполняющие варианты 1–4, реализуют листинг визуализации для своего набора и одну из дополнительных проверок (а–г) в соответствии с вариантом; остальные проверки можно выполнить по желанию.

Алгоритм 2. Анализ сегментационной разметки

Полигон хранится как последовательность вершин (x1, y1), …, (xn, yn), которые обходятся по контуру. Площадь многоугольника в пикселях вычисляется по формуле Гаусса (формуле шнуровки):

$$S = 0{,}5 \cdot |\Sigma\ (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i)|$$

где суммирование ведётся по всем вершинам от 1 до n, а вершина n+1 совпадает с первой. Рамку, охватывающую полигон, получают как минимальные и максимальные значения координат вершин. В листинге ниже также реализован перевод строки аннотации формата YOLO в вершины в пикселях и обратно:

```
import numpy as np



def polygon_area(pts):
    # формула Гаусса (шнуровки)
    x, y = pts[:, 0], pts[:, 1]
    a = np.dot(x, np.roll(y, -1))
    b = np.dot(y, np.roll(x, -1))
    return 0.5 * abs(a - b)



def polygon_to_bbox(pts):
    x_min, y_min = pts.min(axis=0)
    x_max, y_max = pts.max(axis=0)
    return x_min, y_min, x_max - x_min, y_max - y_min



def to_yolo_seg(cls, pts, w, h):
    coords = " ".join(f"{x / w:.5f} {y / h:.5f}" for x, y in pts)
    return f"{cls} {coords}"



def from_yolo_seg(line, w, h):
    # строка разметки -> (класс, массив вершин в пикселях)
    parts = line.split()
    xy = np.array(parts[1:], dtype=float).reshape(-1, 2)
    return int(parts[0]), xy * [w, h]
```

```
pts = np.load("gear_polygon.npy").astype(float)
line = to_yolo_seg(0, pts, 640, 480)
cls, back = from_yolo_seg(line, 640, 480)
print("вершин:", len(back))
print("площадь полигона, пикс.:", round(polygon_area(back), 1))
x, y, w, h = polygon_to_bbox(back)
print("площадь рамки, пикс.:   ", round(float(w * h), 1))
```

Результат на полигоне из 48 вершин (рисунок 4) показывает, насколько рамка грубее полигона: площадь полигона составляет 18 883,6 пикс², а площадь охватывающей его рамки – 28 729,2 пикс². Для проверки значение площади, вычисленное по формуле Гаусса, сравнивают с результатом функции cv2.contourArea: по исходным целочисленным вершинам оба способа дают 18 883,5 пикс². После записи в формат аннотаций и обратного чтения значение составляет 18 883,6 пикс²; различие в 0,1 пикс² вызвано округлением координат до пяти знаков.

Форму объекта описывают показателями, которые вычисляют по вершинам полигона. Коэффициент заполнения рамки показывает, какую часть рамки занимает объект; коэффициент компактности равен единице для круга и уменьшается по мере усложнения контура (P – периметр полигона):

$$K_з = S / (w \cdot h), \quad K_к = 4\pi \cdot S / P^2$$

Центр масс полигона вычисляют по формулам, в которых A – знаковая площадь многоугольника, равная половине суммы в формуле Гаусса без модуля:

$$C_x = \Sigma\,(x_i + x_{i+1}) \cdot (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i) / (6A)$$

$$C_y = \Sigma\,(y_i + y_{i+1}) \cdot (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i) / (6A)$$

Листинг ниже вычисляет периметр, центр масс и перечисленные коэффициенты:

```
import numpy as np



def polygon_perimeter(pts):
    d = pts - np.roll(pts, -1, axis=0)
    return float(np.hypot(d[:, 0], d[:, 1]).sum())



def polygon_centroid(pts):
    x, y = pts[:, 0], pts[:, 1]
    x1, y1 = np.roll(x, -1), np.roll(y, -1)
```

```
    cross = x * y1 - x1 * y
    area = cross.sum() / 2                  # знаковая площадь
    cx = ((x + x1) * cross).sum() / (6 * area)
    cy = ((y + y1) * cross).sum() / (6 * area)
    return cx, cy



pts = np.load("gear_polygon.npy").astype(float)
S = 0.5 * abs(np.dot(pts[:, 0], np.roll(pts[:, 1], -1))
              - np.dot(pts[:, 1], np.roll(pts[:, 0], -1)))
P = polygon_perimeter(pts)
w = pts[:, 0].max() - pts[:, 0].min()
h = pts[:, 1].max() - pts[:, 1].min()

print("коэффициент заполнения рамки:", round(S / (w * h), 3))
compact = 4 * np.pi * S / P ** 2
print("коэффициент компактности:   ", round(compact, 3))
cx, cy = polygon_centroid(pts)
print("центр масс:", [round(float(cx), 1), round(float(cy), 1)])
print("центр рамки:", [float(pts[:, 0].min() + w / 2),
                       float(pts[:, 1].min() + h / 2)])
```

Для полигона шестерни получено: коэффициент заполнения 0,657, коэффициент компактности 0,364, центр масс (359,5; 219,5) при центре рамки (360,0; 219,5). Центр масс и центр рамки у симметричной детали практически совпадают; у несимметричного объекта (например, у дефекта неправильной формы) разница между ними характеризует смещение массы объекта внутри рамки. Периметр и центр масс проверены независимо средствами OpenCV (cv2.arcLength и cv2.moments): значения совпали.

Студенты, выполняющие варианты 5–7, реализуют вычисление площади (первый листинг этого алгоритма) и один из показателей – по варианту: коэффициент заполнения, коэффициент компактности или центр масс.

Алгоритм 3. Разбиение набора на выборки и анализ баланса классов

Алгоритм воспроизводимо делит изображения на обучающую, валидационную и тестовую выборки и подсчитывает, сколько рамок каждого класса попало в каждую выборку. Воспроизводимость обеспечивается фиксацией начального значения генератора случайных чисел (random.seed): при повторном запуске программа выдаёт то же самое разбиение, поэтому результаты экспериментов можно сопоставлять между собой.

```
import random
```

```
from collections import Counter
from pathlib import Path

names = ["gear", "washer", "nut"]
random.seed(42)                          # фиксируем случайность
files = sorted(Path("images").glob("*.jpg"))
random.shuffle(files)

n = len(files)
n_train = int(0.7 * n)
n_valid = int(0.2 * n)
splits = {
    "train": files[:n_train],
    "valid": files[n_train:n_train + n_valid],
    "test": files[n_train + n_valid:],
}

for name, items in splits.items():
    cnt = Counter()
    for img_path in items:
        label = Path("labels") / (img_path.stem + ".txt")
        for line in label.read_text().splitlines():
            cnt[names[int(line.split()[0])]] += 1
    print(f"{name:5s} изображений: {len(items):2d}", dict(cnt))
```

На демонстрационном наборе из 12 кадров программа распределила изображения в соотношении 8/2/2, и в каждой выборке оказались рамки всех трёх классов. На реальном наборе такого равенства может не быть, поэтому наличие всех классов в каждой выборке нужно проверять обязательно: если какой-то класс отсутствует в валидационной или тестовой выборке, оценить качество распознавания этого класса невозможно.

Для кадров из видеозаписей применяют разбиение по роликам. Имя ролика выделяют из имени файла, ролики перемешивают и распределяют по выборкам целиком. Кроме того, в листинге реализован коэффициент дисбаланса Кд:

```
import random
from collections import Counter
from pathlib import Path
```

```
def scene_of(path):
    # video01_0005.jpg -> video01
    return path.stem.split("_")[0]

def split_by_scene(files, ratios=(0.7, 0.2, 0.1), seed=42):
    scenes = sorted({scene_of(p) for p in files})
    random.Random(seed).shuffle(scenes)
    n_train = round(ratios[0] * len(scenes))
    n_valid = round(ratios[1] * len(scenes))
    role = {}
    for i, s in enumerate(scenes):
        if i < n_train:
            role[s] = "train"
        elif i < n_train + n_valid:
            role[s] = "valid"
        else:
            role[s] = "test"
    out = {"train": [], "valid": [], "test": []}
    for p in files:
        out[role[scene_of(p)]].append(p)
    return out

def imbalance(counter):
    # коэффициент дисбаланса: самый частый / самый редкий класс
    return max(counter.values()) / min(counter.values())



demo = [Path(f"video{k // 10 + 1:02d}_{k:04d}.jpg")
        for k in range(100)]
parts = split_by_scene(demo)
print({k: len(v) for k, v in parts.items()})
for k, v in parts.items():
    print(k, sorted({scene_of(p) for p in v}))
cnt = Counter(gear=120, washer=85, nut=12)
print("дисбаланс:", round(imbalance(cnt), 1))
```

При разбиении 100 кадров из 10 роликов программа сформировала выборки из 70, 20 и 10 кадров, причём ни один ролик не оказался сразу в двух выборках; для набора с

числом рамок 120, 85 и 12 коэффициент дисбаланса равен 10,0. Студенты, выполняющие варианты 8–10, реализуют разбиение в режиме, указанном в таблице вариантов.

Порядок выполнения работы

Работа состоит из общей части (выполняется всеми студентами одинаково) и индивидуальной части (выполняется в соответствии с номером варианта). Объект и классы разметки выбираются по таблице вариантов; допускается использовать тот же объект, что и в лабораторных работах №3 и №4.

Общая часть

Шаг 1. Создание проекта в Roboflow. Зарегистрируйтесь на сайте Roboflow и создайте новый проект. Укажите название проекта, группу аннотаций (общее название набора классов) и тип проекта – «Обнаружение объектов» (рисунок 8). Тип проекта определяет, какие инструменты разметки будут доступны.

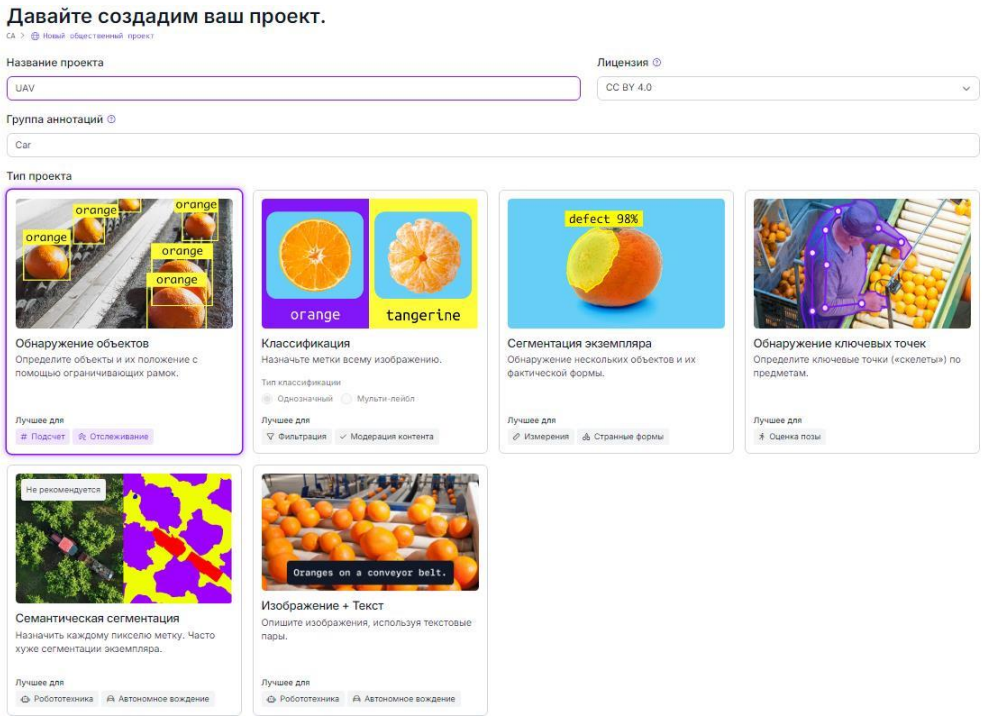


Рисунок 8 – Выбор типа проекта в Roboflow: для общей части работы выбирается «Обнаружение объектов»

Шаг 2. Подготовка и загрузка изображений. Соберите не менее 40 изображений выбранного объекта в разных условиях освещения и ракурсах. Если исходными данными является видео, разделите его на кадры командой ffmpeg (см. выше), сохранив имена файлов с префиксом ролика. Загрузите изображения в проект перетаскиванием каталога в окно загрузки.

Шаг 3. Разметка. Разметьте все изображения ограничивающими рамками, соблюдая правила 1–5. Классы задайте по таблице вариантов; каждого класса в наборе должно быть не менее 10 экземпляров. Сохраните скриншоты не менее трёх размеченных изображений, среди которых должен быть случай перекрытия объектов.

Шаг 4. Разделение на выборки. Задайте соотношение 70/20/10 в окне разделения изображений (рисунок 9), которое Roboflow показывает при загрузке изображений; в зависимости от версии интерфейса то же разбиение может настраиваться и на этапе создания версии набора. Зафиксируйте итоговое число изображений в каждой выборке.

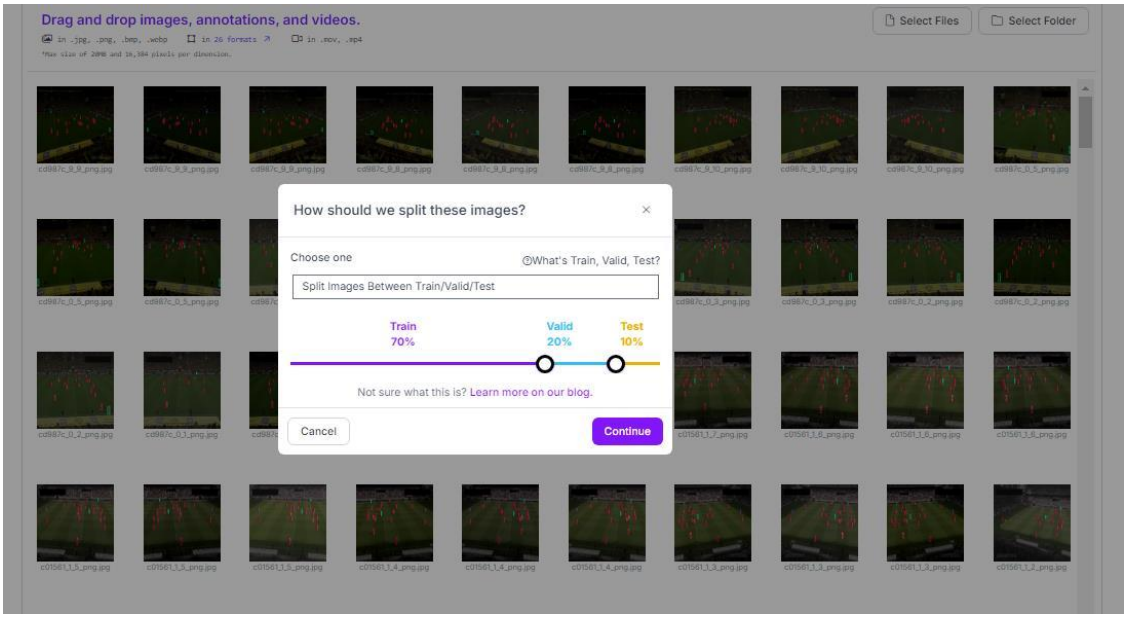


Рисунок 9 – Окно разделения изображений на обучающую, валидационную и тестовую выборки в Roboflow (70 % / 20 % / 10 %)

Шаг 5. Экспорт набора. Экспортируйте версию набора в формате YOLOv8 через интерфейс (кнопка Export Dataset) или программно. Каталог с экспортированным набором содержит папки изображений и аннотаций для каждой выборки и файл data.yaml:

```
!pip install roboflow
from roboflow import Roboflow

rf = Roboflow(api_key="ВАШ_API_КЛЮЧ")
project = rf.workspace("ваш_workspace").project("название_проекта")
version = project.version(1)
dataset = version.download("yolov8")
print(dataset.location)
```

Шаг 6. Первичная проверка разметки. Просмотрите не менее 10 случайных изображений из разных выборок и убедитесь, что все объекты размечены, рамки узкие, а

скрытые объекты обозначены целиком. Найденные ошибки исправьте в Roboflow до выполнения индивидуальной части.

Индивидуальная часть (по варианту)

Шаг 7. В зависимости от номера варианта реализуйте один из трёх алгоритмов:

- варианты 1–4 – алгоритм 1: визуализация разметки и проверка (а), (б), (в) или (г) в соответствии с вариантом;
- варианты 5–7 – алгоритм 2: создайте второй проект типа «Сегментация экземпляров», разметьте полигонами не менее 15 объектов, экспортируйте разметку в формате YOLOv8 и вычислите площадь и показатель по варианту;
- варианты 8–10 – алгоритм 3: выполните разбиение набора в режиме, указанном в таблице вариантов, и подсчитайте число рамок по классам в каждой выборке.

Шаг 8. Примените алгоритм к своему набору данных и оформите результат: для вариантов 1–4 – список найденных замечаний и два-три изображения с восстановленными рамками; для вариантов 5–7 – таблицу значений площади и показателя для каждого полигона; для вариантов 8–10 – таблицу числа изображений и рамок по классам для каждой выборки и значение Кд.

Шаг 9. Проанализируйте результат. Для вариантов 1–4 исправьте найденные ошибки разметки в Roboflow и повторите проверку; для остальных вариантов сделайте вывод о качестве разметки и пригодности набора для обучения. Сформируйте окончательную версию набора для лабораторной работы №6 и сохраните ссылку на неё.

Содержание отчёта

1) Цель работы.
2) Вариант задания, описание производственного объекта и классов разметки.
3) Описание набора данных: источник, число изображений и роликов, условия съёмки.
4) Скриншот проекта в Roboflow с перечнем классов и не менее трёх размеченных изображений.
5) Таблица: число изображений и рамок по классам в обучающей, валидационной и тестовой выборках.
6) Описание применения правил маркировки на примерах (в том числе перекрытых объектов).
7) Листинг индивидуального алгоритма в соответствии с вариантом.
8) Результаты работы алгоритма: вывод программы, таблицы и изображения.
9) Анализ найденных проблем разметки и принятые меры.
10) Ссылка на окончательную версию набора данных для лабораторной работы №6.
11) Выводы по результатам лабораторной работы.

Варианты заданий

Номер варианта определяется преподавателем либо выбирается студентом по согласованию с преподавателем. Общая часть работы (шаги 1–6) выполняется одинаково при любом варианте; индивидуальная часть (шаги 7–9) – в соответствии с последним столбцом таблицы. Названия классов приведены рекомендуемые, их можно изменить по согласованию с преподавателем.

№	Производственный объект	Классы разметки	Индивидуальное задание (шаги 7–9)
1	Детали на конвейере	gear, washer, nut	Алгоритм 1, проверка (а): малые рамки
2	Печатная плата	resistor, capacitor, chip	Алгоритм 1, проверка (б): выход за границы кадра
3	Упаковка и тара	box, bottle, can	Алгоритм 1, проверка (в): пустые файлы разметки
4	Маркировка на изделии	label, barcode	Алгоритм 1, проверка (г): дубликаты рамок
5	Металлическая поверхность с дефектами	scratch, dent, rust	Алгоритм 2: площадь и коэффициент заполнения
6	Пищевой продукт на конвейере	apple, orange, defect	Алгоритм 2: площадь и коэффициент компактности
7	Сварной шов	weld, defect	Алгоритм 2: площадь и центр масс
8	Текстильное полотно	knot, hole, stain	Алгоритм 3: случайное разбиение 70/20/10, таблица рамок по классам
9	Инструмент на складе (не менее 5 роликов)	wrench, hammer, pliers	Алгоритм 3: разбиение по роликам, сравнение со случайным
10	Автомобильная деталь	disc, headlight, mirror	Алгоритм 3: разбиение 80/10/10 и коэффициент дисбаланса

Допускается использовать один и тот же набор изображений для общей и индивидуальной частей; для вариантов 5–7 полигональная разметка выполняется на подмножестве из не менее 15 изображений набора.

Контрольные вопросы

1) Какие данные называются размеченными и почему качество разметки определяет качество обученной модели?

2) Чем различаются разметка для классификации, детектирования и сегментации?

3) Для чего видео разделяют на кадры и почему частоту сохранения кадров выбирают так, чтобы кадры заметно отличались?

4) Назовите правила маркировки изображений. Почему перекрытый объект размечают так, как если бы он был виден полностью?

5) Что означает каждое число в строке аннотации формата YOLO и почему координаты нормируют?

6) Как перевести рамку из координат в пикселях в формат YOLO и обратно?

7) Для чего набор данных делят на обучающую, валидационную и тестовую выборки? Что такое переобучение?

8) Почему кадры одного видеоролика лучше не распределять между разными выборками?

Вопросы 9–11 – по индивидуальному алгоритму (в соответствии со своим вариантом):

9) (Алгоритм 1) Чем пустой файл разметки отличается от отсутствующей разметки на изображении с объектом и в каком случае пустой файл допустим? Какие ошибки разметки не обнаруживаются автоматическими проверками?

10) (Алгоритм 2) Как вычисляется площадь полигона по формуле Гаусса? Что показывают коэффициенты заполнения и компактности и чему равна компактность круга?

11) (Алгоритм 3) Зачем фиксируют начальное значение генератора случайных чисел? Что такое коэффициент дисбаланса и какие меры принимают при его большом значении?

12) В каких случаях достаточно разметки рамками, а в каких необходимы полигоны?

13) Как ошибки разметки отражаются на метриках качества обученной модели?